

Practice #02 — The Prompt Is an Engineering Input

Practice work #02 · 1 point · AI-use level D (AI-integrated, disclosure required) · ~50 min in class + ~60 min at home

Give ONE AI assistant the SAME task FOUR times, changing nothing but the prompt. Test all four outputs against the same six cases. Decide from evidence — not first impressions — which prompt was worth writing.

Write this in whatever language you like. Python, JavaScript, TypeScript, Java, C#, C++, Go, Kotlin, Rust — your choice. The class demo and the ready-made harness are Python, so Python is the *recommended* path, not the required one. Section 3 says exactly what to do instead, and you lose no marks for it.

What must stay the same for everyone: the four prompts, the six test cases, the four returned values and their names, and the requirement that invalid input is rejected with a deliberate error.

1 · Freeze the experiment

An experiment where two things change at once tells you nothing. Fix these three, write them into section 1 of lab-report.md, and do not change them again:

- **One AI assistant** — ChatGPT, Claude, DeepSeek, Gemini, Qwen, whatever you can access.
- **One exact model name** — the one shown in the interface (GPT-5.1, Claude Sonnet 4.5, DeepSeek-V3). "ChatGPT" is not a model name.
- **One language** — your choice, frozen for all four prompts. Prompt A in Python and Prompt B in Java is not an experiment.

Three more rules that decide whether your results mean anything:

- A fresh chat for every prompt. No follow-up questions during the first run.
- Save every response before you touch it. Do not repair the code — not even an import. A repaired output is a destroyed measurement.
- Save each response into code/prompt_a, _b, _c, _d with your language's extension (.py, .js, .java, .go ...).

2 · The four prompts

A — minimal. Send exactly one sentence, nothing else:

Write Python code to analyze student marks.

B — structured context. New chat. Send exactly:

You are a Python developer. Implement analyze_marks(marks, pass_mark=50). Return average, highest, lowest, and pass_rate in a dictionary. Accept marks from 0 to 100; raise ValueError for an empty list, non-numeric values, or out-of-range values. Use no external libraries. Return code plus a short explanation.

C — examples and tests. New chat. Send Prompt B again with this appended:

Example: analyze_marks([40, 60, 80], 50) -> average 60, highest 80, lowest 40, pass_rate 66.67. Include tests for: one mark, decimals, custom pass_mark, empty list, text value, and marks below 0 or above 100. State any remaining assumptions before the code.

D — your combined prompt. New chat. One message of your own, merging everything you learned from A, B and C — including a resolution for every ambiguity you found. Not a conversation: one send, and you take what comes back. Paste its full text into section 5.

Using a language other than Python? Substitute exactly these four tokens

Change nothing else — same wording, same order, same constraints — so the whole class is still sending the same prompt. Record your substituted Prompt B in section 1.

Your language	Ask for this signature	"dictionary" becomes	"ValueError" becomes
Python (recommended)	analyze_marks(marks, pass_mark=50)	dict	ValueError
JavaScript / TS	analyzeMarks(marks, passMark=50)	object / Map	a thrown Error / TypeError
Java	analyzeMarks(List<?>, double)	Map<String, Double>	IllegalArgumentException
C#	AnalyzeMarks(IList<object>, double)	Dictionary<string,double>	ArgumentException
Go	AnalyzeMarks([]any, float64)	map[string]float64	a returned error value
C++	analyze_marks(vector<...>, double)	std::map<string,double>	std::invalid_argument

Two things that do not get substituted. The four returned values keep the names average, highest, lowest, pass_rate exactly, even where your language's convention would camel-case them — a renamed key is a FAIL, and that rule has to mean the same thing for everyone. And "no external libraries" stays: standard library only, in every language.

3 · Test all four the same way

Inputs below are written in Python notation. In your language, use the equivalent: a list or array of numbers, an empty one, one with a text value in it, and one with numbers outside 0–100.

#	Call	Required result
1	<code>analyze_marks([40, 60, 80], 50)</code>	average 60 · highest 80 · lowest 40 · pass_rate 66.67
2	<code>analyze_marks([100], 50)</code>	average 100 · highest 100 · lowest 100 · pass_rate 100
3	<code>analyze_marks([49.5, 50], 50)</code>	average 49.75 · highest 50 · lowest 49.5 · pass_rate 50
4	<code>analyze_marks([], 50)</code>	rejected with a deliberate error (ValueError or your language's equivalent)
5	<code>analyze_marks([40, "60"], 50)</code>	rejected with a deliberate error
6	<code>analyze_marks([-1, 50, 101], 50)</code>	rejected with a deliberate error

Verdicts — use these three words and no others:

PASS	values match (tolerance 0.01), or the required error is raised deliberately
FAIL	it runs but a value is wrong, a required key is missing, or it returns a result where an error was required
ERROR	it does not run, the function is missing, the signature is wrong, or it raises some other exception instead of the specified one

Case 5 in a statically typed language. Java, C#, Go and C++ may refuse to hold a mixed list at all. Take the input as your language's generic type (Object, any, interface{}) or as strings you parse, say which you chose in section 1, and test it. If the AI's code will not even compile with a text value in the list, that is ERROR — record it and say why. A compiler refusing bad input earlier than a runtime check is a real finding, and it comes back in Lesson 07.

Path A — you are using Python

The harness is written for you. Run it once per prompt and paste the complete terminal output of all four runs into section 6 of lab-report.md:

```
python tests/test_analyze_marks.py code/prompt_a.py
python tests/test_analyze_marks.py code/prompt_b.py
python tests/test_analyze_marks.py code/prompt_c.py
python tests/test_analyze_marks.py code/prompt_d.py
```

Do not edit tests/test_analyze_marks.py. It is how your results are checked.

Path B — you are using any other language

The harness cannot run your code, so you write the runner — about twenty lines, and writing it is itself worth the time. It must call the function with each of the six inputs and print, per case: the case number, the value returned or the exception type and message, and your verdict. Finish with a totals line. Commit it as tests/runner.<ext> and paste its real terminal output into section 6.

```
for each of the six cases:
  try:
    result = analyze_marks(input, 50)
    compare each of average, highest, lowest, pass_rate
      numerically, with tolerance 0.01 <- not string comparison
    print case number, the result, and PASS / FAIL
  catch error e:
    print case number, type of e, message, and PASS / FAIL / ERROR
print totals:  n PASS  ·  n FAIL  ·  n ERROR
```

Text output only. Screenshots of an IDE are not accepted — the point is that someone else can re-run it and get the same thing.

4 · Score, conclude, submit

- Score each prompt out of 10 in section 7 (correctness · requirement coverage · verifiability · assumptions stated · noise), then write a 150–200 word conclusion in section 8.
- Be specific or score nothing. "Prompt B was more detailed" is worth zero; "case 3 returned pass_rate=0.0 because it used > instead of >=" is worth everything.
- Fill in AI_USAGE.md, including the exact model name. Undisclosed AI use is an academic-integrity violation, not a style issue.

Submission — the pull request link, pasted into the Week 02 assignment in MS Teams:

```
git checkout main && git pull
git checkout -b week-02          # work inside week-02/ only
git add week-02 && git commit -m "week-02: ..."          # at least 3 commits
git push -u origin week-02
# then open PR week-02 -> main in YOUR repository — DO NOT MERGE IT
```

The practice slide asks you to email screenshots. That does not apply to our group. A merged PR, a repository link instead of a PR link, or work committed to main counts as not submitted.

5 · Grading — 1 point

Criterion	Points
Four prompts in fresh chats; four unedited outputs saved in code/; Prompt D written by you	0.25
Section 6 complete: six cases × four prompts, with real terminal output pasted behind the table	0.25
Scoring table filled; 150–200 word conclusion naming a specific case that changed verdict	0.25
Workflow: week-02 branch, ≥3 commits, open PR, link in Teams, AI_USAGE.md complete	0.25

Your language is never part of the mark. A careful Go submission and a careful Python one score identically; a sloppy Python one does not score better for being Python.

Not accepted: a results table with no terminal output behind it · outputs repaired before testing · all four prompts run in one chat · a conclusion naming no test case · committing to main · a merged or empty PR · a missing AI disclosure.

An honestly reported 2/6 scores higher than a 6/6 nobody can reproduce. If a prompt failed badly, say so — that result is the finding.